

CXXVIII - TP 3

Un petit peu d'interactivité

1 Présentation

Jusqu'à présent, nous avons écrit des codes qui généraient un affichage **statique**. L'objectif de ce TP est de comprendre comment écrire des codes produisant un affichage **dynamiques**, en **interaction** avec l'utilisateur. Pour cela, il est habituel d'utiliser deux fonctions :

- la fonction `draw` : on l'appelle pour dessiner tous les éléments à l'écran. Par exemple dessiner le décors du jeu, les personnages, l'interface (texte des menus, barre de vie etc.).
- la fonction `update` : on l'appelle pour mettre à jour les variables du programme. On l'utilise pour gérer les entrées (touches appuyées par l'utilisateur, boutons de souris cliqués), les déplacements des personnages, les changements d'états...

Une fois que ces deux fonctions sont définies, le programme affiche dynamiquement les **frames** (images, en anglais) les unes après les autres. Pour afficher une frame, le programme appelle d'abord la fonction `update`, puis la fonction `draw`. Par défaut, 30 frames sont affichées chaque seconde (il y a donc au total 30 appels à la fonction `update` et 30 appels à la fonction `draw` par seconde), ce qui permet d'obtenir un affichage dynamique. Il est possible de modifier la valeur par défaut de 30 fps utilisant le paramètre optionnel `fps` dans la fonction `init`.

Code python

```

1  import pyxel
2
3  pyxel.init(128, 128, title="Exemple 1", fps=15)
4
5  COULEURS = list(range(16)) # [0, 1, 2, ..., 15]
6
7  def update():
8      """ Met à jour la logique du programme """
9      # for i in range(len(COULEURS)):
10         #     if COULEURS[i] < 15:
11             #         COULEURS[i] += 1
12         #     else:
13             #         COULEURS[i] = 0
14
15  def draw():
16      """ Affichage à l'écran """
17      # Dessine 16 rectangles verticaux avec des couleurs différentes
18      for i in range(16):
19          pyxel.rect(i * 8, 0, 10, 128, COULEURS[i])
20
21  # Lancement de la boucle principale de Pyxel
22  pyxel.run(update, draw)

```

Question 1. 1. Exécuter le programme ci-dessus. Que constate-t-on ?

2. Décommenter les lignes 10 à 14 de la fonction `update`. Que constate-t-on ?

3. Combien de fois les fonctions `update` et `draw` sont-elles exécutées chaque seconde par ce programme ?

4. Quel est le rôle de la variable `COULEURS` ?

La variable `pyxel.frame_count` est un compteur incrémenté à chaque frame depuis le début de l'exécution du programme. Il représente le nombre d'images écoulées depuis le lancement du programme.

Question 2. 1. En supposant que le programme s'exécute à 15 fps, donner une instruction python permettant de calculer le temps écoulé depuis le début de l'exécution du programme.

2. **Sans modifier** la valeur de 15 fps, modifier le programme précédent de telle sorte que les couleurs des rectangles verticaux soient mis à jour une seule fois par seconde.

Il faut pour cela exécuter les lignes 10 à 14 de la fonction `update` uniquement si le nombre de frames écoulées est un multiple de 15.

2 Gestion des entrées clavier

Le clavier est le moyen privilégié d'interaction avec l'utilisateur. En effet il est courant que ceux-ci utilisent les flèches directionnelles pour se déplacer, ou appuient sur des touches prédéfinies pour effectuer certaines actions : par exemple appuyer sur la touche espace pour sauter, sur la touche R pour lancer un sort, etc.

Pour faire référence à une touche on utilise les variables `pyxel.KEY_SPACE`, pour la touche espace, `pyxel.KEY_R` pour la touche R etc.

Les fonctions `pyxel.btn`, `pyxel.btnp`, et `pyxel.btnr` renvoient `True` respectivement si la touche passée en argument est enfoncée, pressée lors de cette frame, ou relâchée lors de cette frame.

La liste des touches disponibles se trouve dans la [documentation](#).

Code python

```
1 import pyxel
2
3 pyxel.init(128, 128, title="Exemple 2") # 30 fps par défaut
4
5 POSITION = {"x": 64, "y": 64}
6
7 def update_carre():
8     if pyxel.btn(pyxel.KEY_LEFT):
9         POSITION['x'] -= 2
10    if pyxel.btn(pyxel.KEY_RIGHT):
11        POSITION['x'] += 2
12    if pyxel.btn(pyxel.KEY_UP):
13        POSITION['y'] -= 2
14    if pyxel.btn(pyxel.KEY_DOWN):
15        POSITION['y'] += 2
16
17 def update():
18     """ Met à jour la logique du programme """
19     update_carre()
20
21 def draw():
22     """ Affichage à l'écran """
23     pyxel.rect(POSITION["x"], POSITION["y"], 8, 8, 3)
24
25 pyxel.run(update, draw)
```

- Question 3.** 1. Exécuter le code ci-dessus, et appuyer sur les flèches directionnelles. Que constate-t-on ?
2. Modifier le programme de telle sorte que les déplacements se fassent avec le système **ZQSD**.
3. Remplacer les appels à la fonction `pyxel.btn` présents dans la fonction `update_carre` par `pyxel.btnp`. Exécuter le programme et commenter.
4. Remplacer les appels à la fonction `pyxel.btn` présents dans la fonction `update_carre` par `pyxel.btnr`. Exécuter le programme et commenter.
5. Dans le cas où on utilise la fonction `pyxel.btn`, remplacer les `if` présents dans la fonction `update_carre` par `elif`. Exécuter et commenter.

3 Parenthèse : variables globales, objets mutables et immuables

En Python, une **variable globale** est une variable déclarée en dehors de toute fonction ce qui la rend accessible dans l'ensemble du programme. Avec `pyxel`, il n'est pas possible de passer des variables en argument aux fonctions `update` et `draw`. Pour résoudre ce problème il est donc nécessaire d'utiliser des variables globales, accessibles depuis n'importe quelle fonction.

```
Code python
1 var_glob = 64
2
3 def affiche():
4     print(var_glob)
5
6 affiche()
```

⚙️ ➤ Résultat

64

Cependant, leur utilisation est *très subtile*. Considérons la situation suivante : une variable `pos_x` contient l'abscisse de la position du personnage. On souhaite déplacer le personnage de 5 pixel vers la droite : il faut donc incrémenter la variable `pos_x` de 5.

```
Code python
pos_x = 64

def augmente():
    pos_x = pos_x + 5

augmente()
print(pos_x)
```

```
Code python
pos_x = 64

def augmente(x):
    x = x + 5

augmente(pos_x)
print(pos_x)
```

Question 4. Exécuter les codes-ci dessus. Commenter.

Le contenu d'une variable globale peut-être lu depuis une fonction, mais ne peut pas être **modifié**. Pour modifier une variable globale, celle-ci doit être d'un type mutable (par exemple une liste, un dictionnaire, ...) ou bien on peut utiliser le mot-clé `global` (dont l'utilisation est fortement déconseillée, et interdite en terminale).

Remarque. Lorsque la variable `lst` est une liste, il est possible de modifier le *contenu* de `lst` sans modifier `lst` (qui faut toujours référence au même emplacement mémoire) à l'aide d'une instruction `lst[0]` par exemple. Ce type de raisonnement fonctionne avec tous les objets mutables.

Code python

```
pos_x = 64

def augmente():
    global pos_x
    pos_x = pos_x + 5

augmente()
print(pos_x)
```

⚙️ ➤ Résultat

69

Code python

```
pos_x = [64]

def augmente():
    pos_x[0] = pos_x[0] + 5

augmente()
print(pos_x)
```

⚙️ ➤ Résultat

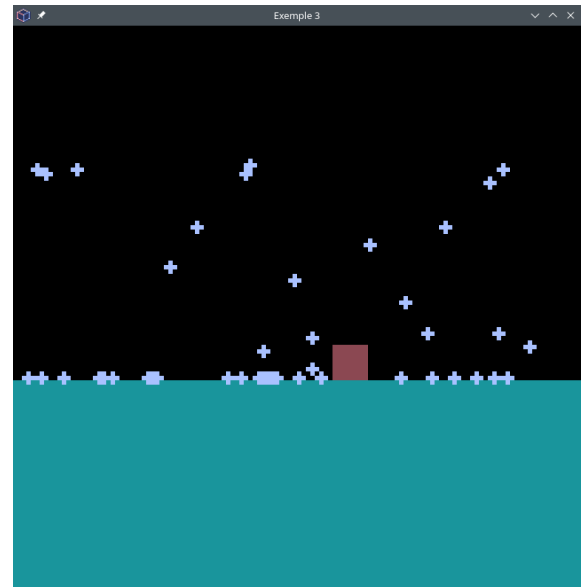
[69]

4 Programmation

L'objectif de ce challenge de programmation est de simuler un personnage se déplaçant sous une averse. On utilisera la fonction `randint` du module `random` (renommée ici `rd`) pour générer aléatoirement la position initiale des gouttes de pluie.

Code python

```
1 import pyxel
2 from random import randint as rd
3
4 FLOOR = 80
5 PERSO = {'x':64, 'y': FLOOR - 8}
6 NB_GOUTTES = 50
7 GOUTTES = [(rd(0, 128), -rd(0, 128))
8             for _ in range(NB_GOUTTES)]
```



Pour cela, on pourra implémenter au choix et dans l'ordre souhaité les fonctions suivantes :

- `draw` : affiche un fond noir et un rectangle de couleur verte afin de symboliser le sol. Affiche un carré à la position donnée par `PERSO` de taille 8×8 pixels. Affiche toutes les gouttes présentes dans la liste `GOUTTES` : des cercles de rayon 1 pixel.
- `update` : met à jour la position du personnage en fonction des touches pressées et met à jour la position de toutes les gouttes.
 - le personnage peut-être déplacé à l'aide des flèches directionnelles. Il ne peut pas sortir de la fenêtre.
 - à chaque frame, toutes les gouttes descendent de 1 pixel. Elles ne peuvent pas descendre plus bas que le sol.

Une fois ce programme de base écrit, vous pouvez implémenter une fonctionnalité simple de votre choix : par exemple mettre sur pause en appuyant sur la touche espace, par exemple générer des gouttes plus grosses que d'autres, par exemple utiliser un fichier `ressources.pyxres` pour afficher un personnage à la place d'un carré (et éventuellement gérer l'orientation du personnage...) etc. Soyez créatifs !